

StegSim Deployment Guide

BCAT-GCAT-Engine/StegSim

Generated: 2026-05-14

ARCHITECTURE CONSTRAINT

No new workflow files. StegSim runs as declared tasks via core-lite-intake.yml. The two permanent workflow files in GCAT-BCAT-Engine/workflows remain the only workflows.

REPO PLACEMENT

```
BCAT-GCAT-Engine/StegSim/           ← this repo (new, parallel to workflows)
  src/stegsim/
    __init__.py
    __main__.py
    model.py
    config.py
    agents.py
    governance.py
    drift.py
    receipts.py
    report.py
    run.py
  tests/
    test_stegsim.py
  examples/
    stale_authority_drift.json
    silent_collapse.json
    recovery_under_lag.json
  tools/
    stegsim_site_export.py
  tools/tasks/
    stegsim_tasks.json              ← merge entries into workflows/tools/tasks/core
  pyproject.toml
```

DEPLOYMENT.md

README.md

INSTALLATION

On your hardware (Ryzen 9 7950X node — primary)

```
# Clone or create the repo
cd ~
git clone https://github.com/BCAT-GCAT-Engine/StegSim # or create new

# Install (v1 deps: numpy + networkx only)
pip install -e ".[dev]"

# Verify install
python -m stegsim --help
```

Verify v1 dependencies (numpy + networkx only)

```
numpy>=1.26
networkx>=3.2
pytest>=7.4 (dev only)
```

DO NOT install PyTorch or Ray for v1. Deferred to v2.

QUICK START

1. Run the built-in demo (1,000 agents, fast)

```
python -m stegsim demo
```

Expected output:

```
[StegSim] Run: demo-stale-authority-001
[StegSim] Initial viability: 0.6500 (healthy)
[StegSim] Step   0: V=0.6500 (healthy) LG=0.000 ...
[StegSim] Step  50: WARNING threshold crossed ...
[StegSim] Final viability: 0.1800 (inadmissible)
```

```
Peak LG disagr: 34.2%
> All dashboards green. System already lost.
```

2. Run the full 10,000-agent scenario

```
python -m stegsim run --config examples/stale_authority_drift.json
```

3. Run all three v1 scenarios

```
python -m stegsim run --config examples/stale_authority_drift.json
python -m stegsim run --config examples/silent_collapse.json
python -m stegsim run --config examples/recovery_under_lag.json
```

4. Run the test suite

```
python tests/test_stegsim.py
# or
python -m pytest tests/ -v
```

Expected: 8/8 tests pass

5. Verify determinism

```
python -m stegsim verify
# Expected: PASS - same seed = same hash
```

INTEGRATE WITH CORE-LITE DISPATCHER

Step 1: Merge task entries

Copy the task objects from `tools/tasks/stegsim_tasks.json` into `GCAT-BCAT-Engine/workflows/tools/tasks/core_lite_tasks.json`.

Add these `task_ids`:

- `stegsim-demo`
- `stegsim-stale-authority`
- `stegsim-silent-collapse`

- stegsim-recovery
- stegsim-verify
- stegsim-test
- stegsim-site-export
- stegsim-sweep (disabled in v1)

Step 2: Add site export tool

Copy `tools/stegsim_site_export.py` to GCAT-BCAT-Engine/workflows/tools/stegsim_site_export.py

Step 3: Run via dispatcher

Actions → core-lite-intake → Run workflow
 task_id: stegsim-demo

Or on your hardware:

```
python core_lite/cli.py run \
  --repo-root . \
  --task-manifest tools/tasks/core_lite_tasks.json \
  --task-id stegsim-demo
```

HARDWARE ASSIGNMENT

Primary node (Ryzen 9 7950X + RTX 4080)

- Main simulation runs
- Full 10,000+ agent scenarios
- Future: v3 LLM-backed auditor agents using RTX 4080

Worker node (i7-12700K + RTX 3070 Ti + RTX 3060 Ti)

- Batch scenario sweeps
- Phase diagram parameter sweeps (v2)
- Repeated seeded runs for variance analysis

v1 capacity (mathematical agents only)

1,000 agents: < 10 seconds
10,000 agents: ~30-60 seconds
100,000 agents: ~5-10 minutes
1,000,000 agents: possible with optimized sampling

OUTPUT STRUCTURE

Each run produces:

```
runs/<run_id>/
  config.json          ← exact parameters used
  receipts.jsonl       ← one receipt per evaluated transition
  metrics.jsonl        ← per-step aggregate metrics
  summary.json         ← run summary with key events
  REPORT.md           ← human-readable explanation
```

SITE INTEGRATION

After any simulation run:

```
python tools/stegsim_site_export.py runs site_data
```

Or via task:

```
task_id: stegsim-site-export
```

This writes `site_data/stegsim_summary.json`.

Add to Site pages:

```
fetch('./data/stegsim_summary.json')
  .then(r => r.json())
  .then(data => {
    document.getElementById('stegsim-headline').textContent = data.headline;
    // render data.runs as table
  });
```

v2 UPGRADE PATH (when ready)

To enable phase diagram sweep:

1. Enable `stegsim-sweep` task in `core_lite_tasks.json` (change `"enabled": false` to `true`)
2. Run: `task_id: stegsim-sweep`
3. Outputs to `runs/sweep/phase_diagram.jsonl`
4. Visualize with `matplotlib` or `plotly`

To enable hybrid agents:

1. Set `"enable_hybrid_agents": true` in `config`
2. Implement `HybridAgent` subclasses in `agents.py`
3. PyTorch install required at that point

VERIFICATION CHECKLIST

```
[ ] pip install -e ".[dev]" succeeds with numpy + networkx only
[ ] python -m stegsim demo completes
[ ] REPORT.md exists in runs/demo-stale-authority-001/
[ ] 8/8 test_stegsim.py tests pass
[ ] python -m stegsim verify prints PASS
[ ] stegsim-demo task runs via core-lite-intake dispatcher
[ ] site_data/stegsim_summary.json produced by stegsim-site-export task
[ ] Two workflow files only in .github/workflows/
```

TRANSITION PERIODIC TABLE — v3 SEED

The `model.py` already defines `TRANSITION_CLASSES`:

```
T-000: idle
T-101: informational_mutation
T-201: resource_transfer
T-221: resource_redistribution
T-301: trust_update
```

T-311: trust_fragmentation	← irreversible
T-401: authority_assertion	
T-402: authority_escalation	← irreversible, critical
T-501: policy_update	
T-511: policy_lag_accumulation	
T-550: irreversible_collapse	← terminal
T-601: recovery_action	

T-550 corresponds to the FAIL_CLOSED region of the GOD tetrahedron. v3 will formalize these classes, add consequence semantics, and link each class to a GOD geometry region.

This data is already being recorded in receipts.jsonl via proposal_id. v3 adds: classify each receipt by transition class, compute class-specific admission rates, and render the periodic table as a Site page.

THE CORE CLAIM

Every run of StegSim produces executable evidence for:

A system can remain locally compliant while becoming globally ungovernable.

The metric that proves it: local_global_disagreement_rate. The receipt that records it: every DENY where local_policy_valid=true. The report that explains it: REPORT.md.